Dynamic document creation using RSP

Henrik Bengtsson

June 27, 2022

Abstract

An important part of a statistical analysis is to document the analysis and its results. A common approach is to build up an R script as the the analysis progresses. This script may generate image files and tables that are later inserted manually into, say, a LaTeX report. This strategy works alright for small one-off analyzes, whereas for larger and repetitive analyzes an automatic report generator is to prefer.

In this document we describe the RSP markup language and explain how it can be embedded in virtually any text-based source document which then may be compile into a final document. We will describe all of the RSP constructs available and illustrate how they can be used for plain text as well as for LaTeX documents.

RSP provides an easy, yet powerful and flexible way for generating reports and other documents in R. With a single R command it is possible to compile an RSP-embedded LaTeX, Sweave or knitr document into a read-to-view PDF file to name a few examples. As an example, we show how to produce this very document from an RSP-embedded LaTeX file. RSP can also be used for R package vignettes, which has become particularly simple since R v3.0.0.

Furthermore, because RSP is a so called content-independent markup language, it can be used to produce documents of any kind, e.g. plain text, LaTeX, Sweave, knitr, AsciiDoc, Markdown, HTML, XML, SVG, and even Javascript, R scripts, tab-delimited data files and so on. More over, with RSP it is possible to use literate programming constructs that are not possible in Sweave, e.g. looping over a mix of R source code and LaTeX text blocks.

Keywords: RSP markup language, literate programming, reproducible research, report generator, Sweave, knitr, brew, noweb, TeX, LaTeX, Markdown, AsciiDoc, reStructuredText, Org-Mode, HTML, PDF

This vignette is distributed as part of the R.rsp package, which is available on CRAN.

Contents

1 Introduction to RSP **3**
- 1.1 rcat() - the RSP version of cat() . . . 3
- 1.2 rsource() - the RSP version of source() . . . 3
- 1.3 rfile() - start-to-end processing of RSP documents . . . 3
  - 1.3.1 Automatic postprocessing . . . 4

2 The RSP markup language **4**
- 2.1 Comments (<%--⟨anything⟩--%>) . . . 5
- 2.2 Escaping RSP start and end tags (<%% and %%>) . . . 6
- 2.3 RSP code expressions (<%...%>) . . . 6
  - 2.3.1 Evaluating code (<%⟨code⟩%>) . . . 6
  - 2.3.2 Inlining values of variables and expressions (<%=⟨code chunk⟩%>) . . . 7
  - 2.3.3 Iterating over a mixture of RSP constructs and text blocks . . . 7
  - 2.3.4 Templates - reusing a mixture of RSP and text blocks . . . 8
- 2.4 Trimming whitespace for RSP constructs . . . 9
  - 2.4.1 Trimming of lines with only non-text RSP constructs . . . 9
  - 2.4.2 Manually controlling trailing whitespace and line breaks (-%> and +%>) . . . 9
- 2.5 Preprocessing directives (<%@...%>) . . . 10
  - 2.5.1 Including text and file contents . . . 10
    - 2.5.1.1 Including file contents (<%@include file="⟨pathname|URL⟩"%>) . . . 11
    - 2.5.1.2 Including plain text (<%@include content="⟨content⟩"%>) . . . 11
  - 2.5.2 Document metadata (<%@meta ...%>) . . . 11
    - 2.5.2.1 Setting metadata via R vignette metadata . . . 12
  - 2.5.3 Preprocessing variables (<%string ...%>) . . . 12
  - 2.5.4 Conditional inclusion/exclusion (<%@if ...%>) . . . 13
    - 2.5.4.1 If-then-else aliases (<%@ifeq ...%>) . . . 14
- 2.6 Advanced . . . 14
  - 2.6.1 Compiling a standalone RSP document from a preprocessed but not evaluated RSP document template . . . 14

3 Solutions specific to the R environment **14**
- 3.1 Working with figures . . . 14
  - 3.1.1 Creating image files . . . 14
  - 3.1.2 Including image files in Markdown . . . 15
  - 3.1.3 Including image files in HTML . . . 15
  - 3.1.4 Including image files in LaTeX . . . 16
- 3.2 Including R code . . . 17
  - 3.2.1 Evaluating and capturing expression and output (<%= withCapture (⟨code chunk⟩)%>) 17
  - 3.2.2 Future directions for including code . . . 18

4 History of RSP **18**

1 Introduction to RSP

The RSP markup language makes it possible to interweave text and source code in a compact and powerful way for the purpose of generating text of any format. An example of an RSP-embedded string is

```
"A random integer in [1,100]: <%=sample(1:100, size=1)%>\n"
```

This string contains three RSP constructs, namely a *plain text string* `"A random integer in [1,100]: "`, an *RSP expression* `<%=sample(1:100, size=1)%>`, which here contains R code, and a second text string `"\n"`. The idea of RSP is to translate this into source code, which somewhat simplified would be

```
cat("A random integer in [1,100]:")
cat(sample(1:100, size=1))
cat("\n")
```

By evaluating this code a text string is outputted, e.g.

```
A random integer in [1,100]: 77
```

1.1 rcat() - the RSP version of cat()

Using the RSP engine of the *R.rsp* package[1], the above RSP string can be processed and outputted as

```
> library("R.rsp")
> rcat("A random integer in [1,100]: <%=sample(1:100, size=1)%>\n")
A random integer in [1,100]: 77
```

As explained further in Section 2, the RSP construct `<%=`⟨`code chunk`⟩`%>` causes the R expression '⟨`code chunk`⟩' to be evaluated and its value to be inserted into the text (replacing the RSP construct). To process the RSP string without outputting the result, but instead returning it in a character string, do

```
> s <- rstring("A random integer in [1,100]: <%=sample(1:100, size=1)%>\n")
> print(s)
[1] "A random integer in [1,100]: 43\n"
```

1.2 rsource() - the RSP version of source()

Consider instead the case where the above RSP string is stored in the text file `random.txt.rsp`. Analogously to how `source()` runs an R script, `rsource()` runs an RSP document, e.g.

```
> rsource("random.txt.rsp")
A random integer in [1,100]: 14
```

1.3 rfile() - start-to-end processing of RSP documents

As an alternative to `rsource()`, an RSP file can be processed using `rfile()`, which instead of outputting to standard output ("displaying on screen") writes the output to file. For instance,

```
> rfile("random.txt.rsp")
```

generates a plain text file (`random.txt`) that contains

[1] To install the *R.rsp* package, call `install.packages("R.rsp")` at the R prompt.

```
A random integer in [1,100]: 96
```

The filename of the output is the same as the RSP file with the file extension dropped. To try this example yourself, do

```
path <- system.file("exData", package="R.rsp")
rfile("random.txt.rsp", path=path)
```

which generates `random.txt` in the current directory. To process the same RSP file into a string (without generating a file), do

```
path <- system.file("exData", package="R.rsp")
s <- rstring(file="random.txt.rsp", path=path)
cat(s)
```

The latter example illustrates how `rstring()` also can load RSP strings from either a file (or a URL or a connection) as an alternative to taking plain strings as input.

1.3.1 Automatic postprocessing

For RSP documents that output certain file formats such as LaTeX, Sweave, and knitr, further processing of the RSP file output is often necessary before obtaining the final RSP product, e.g. a PDF file. The `rfile()` function will, based on the content type of the generated RSP output, detect if further processing ("postprocessing") is possible. If so, it is postprocessed to arrive at the final product. Currently, postprocessing to generate PDF and HTML output is directly supported for LaTeX, Markdown, AsciiDoc, Sweave and knitr documents. For instance, consider an RSP-embedded LaTeX document `report.tex.rsp`. When compiling it by

```
> rfile("report.tex.rsp")
```

the `rfile()` method will (i) parse and evaluate the RSP-embedded LaTeX document resulting in a LaTeX document (`report.tex`), but in addition it will also (ii) compile the LaTeX document into a PDF (`report.pdf`). To disable RSP postprocessing, specify argument `postprocess=FALSE` when calling `rfile()`. To try postprocessing yourself, compile this very document by

```
> path <- system.file("doc", package="R.rsp")
> rfile("Dynamic_LaTeX_reports_with_RSP.tex.rsp", path=path)
```

The generated PDF (`Dynamic_LaTeX_reports_with_RSP.pdf`) will be available in the current directory of R (see `getwd()`). It is also possible to compile online RSP documents, e.g.

```
> url <- "https://raw.githubusercontent.com/HenrikBengtsson/R.rsp/
          master/vignettes/Dynamic_document_creation_using_RSP.tex.rsp"
> rfile(url)
```

which produces a PDF of the current developer's version of this vignette.

2 The RSP markup language

An *RSP-embedded text document*, or short an *RSP document*, is a text document (string or file) that contains a set of *RSP constructs*, which each is defined by a pair of RSP start (`<%`) and end (`%>`) tags. There are three main types of RSP constructs:

1. *RSP comments* (`<%--...--%>`),

2. *RSP preprocessing directives* (`<%@...%>`), and
3. *RSP code expressions* (`<%...%>`).

Everything outside of RSP constructs is referred to as *RSP text*, or short just as '*text*'. When an RSP document is *processed* ("compiled"), it is first (i) *parsed* where comments are dropped and preprocessing directives are processed (and replaced by either text or RSP code expressions), then (ii) *translated* into plain R code, which when (iii) evaluated generates an *RSP product* (as a string or an output file).

Remark: An RSP comment may contain anything, including other RSP constructs. An RSP preprocessing directive may contain RSP comments (which are dropped), but not RSP code expressions. An RSP code expression may contain RSP comments (dropped) and RSP preprocessing directives (processed before parsing the RSP code expression), but not other RSP code expressions.

The simplest RSP document possible is a string that contains none of the above RSP constructs, just a single RSP text, which will be outputted "as is"[2] in the outputted RSP product. A slightly more elaborate example is the RSP string

```
"A random integer in [1,100]: <%=sample(1:100, size=1)%>\n"
```

which is parsed into the following sequence of RSP constructs: (i) text '`A random integer in [1,100]: `', (ii) RSP code expression '`<%=sample(1:100, size=1)%>`', and (iii) text '`\n`'. When this sequence is later evaluated, the two text components are inserted into the RSP product "as is", whereas for the RSP code expression its contained R code is evaluated and the return value is inserted into the RSP product as text.

2.1 Comments (<%--⟨anything⟩--%>)

The RSP markups `<%--⟨anything⟩--%>`, `<%---⟨anything⟩---%>` and so on represent RSP comments, which can be used to drop large sections of an RSP document[3]. Unlike other RSP constructs, an RSP comment may contain other RSP constructs (also incomplete ones) including RSP comments. In order to nest RSP comments, any nested comment must use a different number of hyphens compared to the surrounding RSP comment[4]. For example, the following RSP document

```
<%-- This is an RSP comment that will be dropped --%>
You can write a paragraph and drop a large portion of it using
<%--- This comment contains both regular RSP expressions
There are <%=n%> red <%=type%>s
<%-- as well as another RSP comment --%>
which is nested. ---%>RSP comments.
```

is from the parser's point of view equivalent to the following RSP document

```
You can write a paragraph and drop a large portion of it using
RSP comments.
```

which produces

[2]Except for escaped RSP start and end tags as explained in Section 2.2.

[3]An RSP comment may can contain *anything* because it is dropped by the RSP parser at the very beginning before parsing the text and the other RSP constructs.

[4]The reason for needing a different number of hyphens in nested RSP comments is because RSP comments are *non-greedy*, that is, anything between (and including) the `<%--` and the *first* following `--%>` will be dropped.

```
You can write a paragraph and drop a large portion of it using
RSP comments.
```

Remark: The number of prefix and suffix hyphens in an RSP comment must match, i.e. `<%--- A comment --%>` is *not* recognized as an RSP comment.

Remark: As explained in Section 2.4, *all* RSP constructs with a suffix hyphen causes any following whitespace including the first following line break to be trimmed, if and only if no other characters appear after the construct. This means that all RSP comments automatically trim such empty text. This explains why there are no empty lines after dropping the RSP comments in the above example.

Remark: Trailing whitespace can be trimmed from any line by adding `<%----%>` at the end of that line. For convenience, one can also use `<%-%>`, which is a short form for `<%----%>`.

2.2 Escaping RSP start and end tags (<%% and %%>)

To include '`<%`' and '`%>`' in the text of the RSP product they need to be escaped so that the RSP parser does not recognize them as being part of an RSP construct. This can be done as '`<%%`' and '`%%>`', respectively. For instance,

```
> rcat("A random integer in [1,100]: <%%=sample(1:100, size=1)%%>\n")
A random integer in [1,100]: <%=sample(1:100, size=1)%>
```

To clarify this further, consider the output of the following steps

```
> a <- "A random integer in [1,100]: <%%=sample(1:100, size=1)%%>\n"
> b <- rstring(a)
> rcat(b)
A random integer in [1,100]: 8
```

Remark: The string '`<%%>`' is interpreted as an escaped '`<%`' followed by a '`>`' and *not* as an empty RSP code expression.

2.3 RSP code expressions (<%...%>)

2.3.1 Evaluating code (<%⟨code⟩%>)

The RSP markup `<%⟨code⟩%>` evaluates the source code (without displaying it in the output document). For instance,

```
<%
n <- 3;  # Comments are kept
type <- "horse"
%>
```

evaluates the code such that `n == 3` and `type == "horse"` afterward.

Note that with this construct it is possible to insert incomplete code expression and completed it in a later RSP code block. For example,

```
Counting:<% for (i in 1:3) { %> <%=i%><% } %>.
```

For more details on this, see Section 2.3.3.

Remark: The source code specified in an RSP code expression must have at least one character (which may be a blank). Notably, `<%%>` is *not* an RSP code expression with reasons is explained in Section 2.2.

2.3.2 Inlining values of variables and expressions (<%=⟨code chunk⟩%>)

The RSP markup <%=⟨code chunk⟩%> evaluates the code chunk (without inserting the code itself into the document) and inserts the character representation[5] of the returned object. For instance,

```
Today's date is <%=Sys.Date()%>
```

would produce the string 'Today's date is 2022-06-30'. If inlining a vector of values, they are all pasted together without a separator. For example,

```
The letters of the alphabet are '<%=LETTERS%>'
```

produces "The letters the alphabet are 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'". To separate the elements with commas, use <%=paste(LETTERS, collapse=", ")%>. Alternatively, use <%=hpaste(LETTERS)%> to output 'A, B, C, ..., Z'[6].

Remark: When inlining results, the ⟨code chunk⟩ must be a complete and valid R expression, otherwise it would not be able to evaluate it and return the result.

2.3.3 Iterating over a mixture of RSP constructs and text blocks

A useful feature of RSP is that it is possible to use RSP constructs that span multiple code and text blocks. For instance, the following will iterate over a set of text and code blocks

```
The <%=n <- length(letters)%> letters in the English alphabet are:
<% for (i in 1:n) { %>
  <%=letters[i]%>/<%=LETTERS[i]%><%=if(i < n) ", "%>
<% } %>.
```

which generates[7]: 'The 26 letters in the English alphabet are: a/A, b/B, c/C, d/D, e/E, f/F, g/G, h/H, i/I, j/J, k/K, l/L, m/M, n/N, o/O, p/P, q/Q, r/R, s/S, t/T, u/U, v/V, w/W, x/X, y/Y, z/Z .'

A more complex example is where one wish to generate a report on human genomic data across all of the 24 chromosomes and where the same type of analysis should be repeated for each chromosome. With RSP markup, this can be achieved by an outer loop over chromosomes, as illustrated by

```
<% for (chromosome in 1:24) { %>
\section{Chromosome <%=chromosome%>}
...
A mix of RSP and text blocks constituting
the analysis of the current chromosome.
...
<% } # for (chromosome ...) %>
```

Remark: Note that there exist no corresponding construct in noweb-style markup languages, e.g. Sweave. Instead, contrary to RSP, Sweave requires that each of the code chunks contains a complete R expression. This means that, in terms of the above example, in Sweave it is not possible to begin a for loop in one code

[5] In R, the *character representation* of an object `x` is what generic function `rpaste(x)` gives. The default returns `paste0(as.character(x), collapse="")`, but may be refined and customized for particular classes in the RSP document.

[6] The `hpaste()` function of *R.utils* provides "human-readable" pasting of vectors.

[7] Of course, in this particular case, the above for-loop can be replaced by <%=paste(letters, LETTERS, sep="/", collapse=", ")%>.

chunk and end it in a succeeding one. This has to do with the fundamentally different way RSP and Sweave documents are processed. If using Sweave, one solution is to use RSP markup in the Sweave document, process it via RSP to generate a pure Sweave document, which can then in turn be processed using the Sweave engine to get a TeX file and eventually the final PDF document. Alternatively, one can use `rfile()` to complete all these steps at once, e.g. `rfile("report.Rnw.rsp")` will generate `report.pdf` as well as the intermediate `report.Rnw` and `report.pdf` files. This illustrates the power of the RSP language and how it can be used as a preprocessor of any other literate programming language.

2.3.4 Templates - reusing a mixture of RSP and text blocks

Sometimes rather similar paragraphs of text, tables, or figures are used throughout a document with only minor differences. Instead of manually cut'n'pasting the same pieces to other places of the document over and over, it is more robust and much easier to setup a template function which can then used in place.

Because of the nature of the RSP language, setting up a template is as simple as wrapping the mixture of RSP and code blocks (Section 2.3.3) in a function definition. For example, assume you wish to reuse the following RSP passage multiple times with different values of `n` in a LaTeX document

```
The sum of $x=<%=hpaste(1:n, abbreviate="\\ldots")%>$ is <%=sum(1:n)%>.
```

The solution is to wrap it up in a *template* function

```
<% myTemplate <- function(n, ...) { %>
The sum of $x=<%=hpaste(1:n, abbreviate="\\ldots")%>$ is <%=sum(1:n)-%>.<%-%>
<% } # myTemplate() %>
```

Recall that there is no limitation in how many text and RSP blocks you can use. Note also how we use an empty RSP comment (`<%-%>`) at the end of the template function. That is used in order to escape the trailing line break (Section 2.1). Without the trailing RSP comment, the final document would contain a whitespace after the period (and before the closing single-quote in the previous sentence). The above template function can now be used by calling it within an RSP code block as

```
<% myTemplate(n=3) %>
```

which produces 'The sum of $x = 1, 2, 3$ is 6.'. Naturally, a template function can be reused any number of times. For example, the following RSP-embedded LaTeX block

```
\begin{itemize}
<% for (ii in c(3,5,10,100)) { %>
 \item <% myTemplate(n=ii) %>
<% } # for (ii ...) %>
\end{itemize}
```

produces

- The sum of $x = 1, 2, 3$ is 6.
- The sum of $x = 1, 2, 3, 4, 5$ is 15.
- The sum of $x = 1, 2, 3, \ldots, 10$ is 55.
- The sum of $x = 1, 2, 3, \ldots, 100$ is 5050.

Remark: When using templates, use only `<% ... %>` and not `<%= ... %>`. The reason for this is that the way templates are setup they already output their contents/results to the generated RSP product.

2.4 Trimming whitespace for RSP constructs

Because of readability of the RSP source document, RSP constructors are often placed on lines separate from the text blocks but also other RSP constructs. Depending on how whitespace and line breaks (as introduced by the above coding style) are handled in the output format, this may or may not affect how the output document renders. For instance, line breaks explicitly introduced in an RSP source document will make the an outputted plain text document look different whereas single line breaks will not affect how a LaTeX document is compiled.

2.4.1 Trimming of lines with only non-text RSP constructs

The whitespace before and after RSP constructs that are on their own lines without no other non-whitespace text will *always* have their surround whitespace trimmed off. They will also have the first following line break removed, unless it is manually controlled as describe in the next subsection. This automatic trimming of "stand-alone" RSP constructs enhanced readability of RSP source documents without introducing unnecessary whitespace and line breaks. Consider the following example and think about how whitespace and line breaks are removed or kept:

```
You don't have to worry too much about whitespace, e.g. the
  <%
     s <- "will have its surrounding whitespace"
  %>
above RSP expression <%=s%>
trimmed off as well as its trailing line break.
```

which produces

```
You don't have to worry too much about whitespace, e.g. the
above RSP expression will have its surrounding whitespace
trimmed off as well as its trailing line break.
```

2.4.2 Manually controlling trailing whitespace and line breaks (-%> and +%>)

Also for RSP constructs that are not surrounded by only whitespace, it is possible to control how whitespace and line breaks following RSP constructs are trimmed. This is done via so called *RSP end tag specifications*. Those apply to all types of RSP constructs. By adding an *end tag hyphen* at the end of an RSP construct (i.e. `-%>`), all following whitespace (space and tabs) including the *first* line break (the newline) is dropped ("trimmed"), if and only if there are no other characters following the RSP construct. How this works is illustrated in the following example:

```
> rstring("A random integer in [1,100]: <%=sample(1:100, size=1)%>\n")
[1] "A random integer in [1,100]: 48\n"
> rstring("A random integer in [1,100]: <%=sample(1:100, size=1)-%>\n")
[1] "A random integer in [1,100]: 48"
> rstring("A random integer in [1,100]: <%=sample(1:100, size=1)-%> \t \n\n\n")
[1] "A random integer in [1,100]: 48\n\n"
```

To disable the trimming of trailing whitespace and line breaks, add an *end tag plus sign*, i.e. '`+%>`'. For instance,

```
> rstring("abc\n<%='DEF'%>\nGHI")
[1] "abc\nDEF\nGHI"
> rstring("abc\n<%='DEF'-%>\nGHI")
[1] "abc\nDEFGHI"
> rstring("abc\n<%='DEF'+%>\nGHI")
[1] "abc\nDEF\nGHI"
```

2.5 Preprocessing directives (<%@...%>)

An *RSP preprocessing directive*, or short an *RSP directive*, is an RSP construct that starts with an '@' followed by a *directive* and zero or more *named arguments*, e.g. `<%@include file="random.tex.rsp"%>`. RSP directives can for instance be used to include the content of another file or conditionally include/exclude parts of the existing document. RSP preprocessing directives facilitate modularization of RSP documents such that the same RSP module can be reused in many places, which in turn minimizes and often eliminates the manual and often error-prone process of cut-and-pasting content between documents. RSP preprocessing directives are processed during the parsing of the RSP document. This is done *after* RSP comments have been dropped and *before* RSP code expressions are parsed. This means that

- RSP comments can be used to commenting out and hence exclude RSP preprocessing directives, e.g. `<%--- <%@include file="random.tex.rsp"%> ---%>`.
- RSP preprocessing directives may insert new RSP code expressions as if they where part of the original RSP document, e.g. `<%@include file="random.tex.rsp"%>`. Any RSP comments inserted will be dropped as expected. If other RSP directives are inserted, they are processed immediately.
- After being processed, only RSP text and RSP code expressions remains (no RSP preprocessing directives). Notably, RSP code expressions *do not see or have access* to RSP preprocessing directives or any of their assigned variables.

The value of an RSP directive argument is parsed as a *GString*[8]. A GString is a regular string that optionally may contain bash-like syntax for inserting values of system environment variables. Note how this substitution is indepent of the R language. For convenience, when processing an RSP document using the RSP parser of the *R.rsp* package, GString also looks for a matching variables elsewhere. Notably, it searches (i) among the variables in the R workspace, (ii) among the R options, and then (iii) among the system environment variable.[9] Use of GStrings will be illustrate further in below sections.

Remark: Variables set in RSP code expressions *cannot* be used in RSP directives, because they are only assigned after the RSP preprocessing directives have been processed.

Remark: All RSP directive arguments (also known as "attributes") must be named with names having the correct case and not being abbreviated. For instance, `<%@include file="random.tex.rsp"%>` is correct whereas `<%@include "random.tex.rsp"%>`, `<%@include File="random.tex.rsp"%>` and `<%@include f= "random.tex.rsp"%>` are not.

Remark: The formal syntax for an RSP preprocessing directive is `<%@`⟨`directive`⟩ `[`⟨`name`⟩`="`⟨`value`⟩`"]*%>`. An argument name may consist of alphanumeric characters and underscores where the first being a letter or an underscore. The argument values are parsed as strings and must be quoted either using matching single or double quotes.

Remark: Just like RSP comments, RSP directives are designed to be independent of the programming language. In other words, by design these constructs are not relying on the R language. Another way look at it is that the processing of RSP comments and RSP directives may be done by an *RSP processor* written in a different language than R, e.g. Python or C.

2.5.1 Including text and file contents

It is possible to include the contents of other text and RSP documents. These can be either specified as text strings (argument '`content`') or as files (argument '`file`'), which may be either a file on the file

[8] GStrings can be processed in R by the `gstring()` function in the *R.utils* package.

[9] Although the RSP parser of the *R.rsp* package also looks for GString variable in the R environment and its options, it does not mean that these variables are dependent on the R language. The substituted value will always be interpreted as a character string. An RSP parser implemented in Python could take a similar approach.

system or URL/document online.

2.5.1.1 Including file contents (<%@include file="⟨pathname|URL⟩"%>)

To include the content of a text file "as is" during preprocessing, use the '`file`' argument. This may be used to include a local file or a file online (specified as a URL). For instance, to include the content of text file '`randoms.txt`' as is into the RSP document, use

```
<%@include file="randoms.txt"%>
```

To include the *RSP preprocessed* content of RSP file '`random.txt.rsp`', use

```
<%@include file="randoms.txt.rsp"%>
```

Any file pathname is *relative to the directory* of the (host) RSP document that contains the include directive. For instance, consider an RSP file '`rsp/overview.txt.rsp`' and a text file '`inst/NEWS`'. Next, if an RSP document includes the RSP file as

```
<%@include file="rsp/overview.txt.rsp"%>
```

then that RSP file can in turn include the text file as

```
<%@include file="../inst/NEWS"%>
```

Remark: When including files from the file system, only *relative pathnames* are allowed[10]. If an RSP document could use absolute pathnames, then it would only compile on a particular computer system/user account and would have to be altered in order to compile elsewhere. Note that it is always possible to use file system links[11] to link to files elsewhere on the file system without having to copy them the directory of the source RSP document (or a subdirectory therein).

2.5.1.2 Including plain text (<%@include content="⟨content⟩"%>)

Although not commonly needed, it is, as an alternative to contents of files, possible to include contents of strings. For example, the template RSP document

```
The R_HOME environment variable was '<%@include content="${R_HOME}"%>'
at the time when this document was preprocessed.
```

results in an preprocessed RSP document consisting of

```
The R_HOME environment variable was '/usr/lib/R'
at the time when this document was preprocessed.
```

2.5.2 Document metadata (<%@meta ...%>)

It is possible to assign non-visible metadata to RSP documents. This metadata can be set and retrieved by RSP preprocessing directives. To set an RSP metadata variable, use directive `<%@meta name="⟨name⟩" content="⟨content⟩"%>`. To insert RSP metadata into the RSP output, use directive `<%@meta name="⟨name⟩"%>`. For example, in LaTeX you can do

[10] The RSP compiler in the *R.rsp* package will given an error if it detects an absolute pathname.

[11] The `createLink()` function in *R.utils* provides a platform-independent way for creating file system links.

```
<%@meta name="title" content="My Report"%>
<%@meta name="author" content="John Doe"%>
<%@meta name="keywords" content="statistics, report"%>

% Set the PDF metadata
\usepackage[hidelinks]{hyperref}
\hypersetup{
  pdfauthor={<%@meta name="author"%>},
  pdftitle={<%@meta name="title"%>},
  pdfkeywords={<%@meta name="keywords"%>}
}

\title{<%@meta name="title"%>}
\author{<%@meta name="author"%>}
\keywords{<%@meta name="keywords"%>}
```

Note how this sets both the displayed metadata as well as the non-visible PDF metadata.

Remark: Metadata set in the main document is visible also to "child" RSP documents included via `<%@include ⟨file⟩="⟨pathname|URL⟩"%>`. Vice verse, metadata set in a child document are visible (assigned) to the "parent" document.

Remark: For convenience, `<%@meta ⟨name⟩="⟨content⟩"%>` is short syntax for setting metadata.

2.5.2.1 Setting metadata via R vignette metadata

The R framework uses a LaTeX-flavored markup for specifying metadata in vignettes, e.g. `\VignetteIndexEntry{My Report}` and `\VignetteKeyword{statistics}`. It is possible to "import" this metadata information via `<%@meta content="⟨R vignette metadata markup⟩" language="R-vignettes"%>`. For instance,

```
<%@meta content="
  DIRECTIVES FOR R:
  %\VignetteIndexEntry{My Report}
  %\VignetteKeyword{statistics}
  %\VignetteKeyword{report}
  %\VignetteAuthor{John Doe}
" language="R-vignette"%>

\title{<%@meta name="title"%>}
\author{<%@meta name="author"%>}
\keywords{<%@meta name="keywords"%>}
```

will import the three R `\Vignette⟨Name⟩{}` "commands" into RSP metadata variables 'title', 'keywords' and 'author'. As illustrated, any content that is not recognizes as R vignette metadata, or is of unknown markup, is ignored, which means that it is possible to include "anything" part of the '`content`' attribute. Note that this construct will (i) import the metadata into the RSP document at the same time as (ii) the R vignette mechanism can utilize it. This eliminates the need for reproducing the same information in the RSP source document.

2.5.3 Preprocessing variables (`<%string ...%>`)

In addition to metadata, it is possible to define, set and get custom variables available during the preprocessing of the RSP document. To set an RSP preprocessing *string* variable, use directive `<%@string name="⟨name⟩" content="⟨content⟩" default="⟨default⟩"%>`, where '`default`' is optional and the value

used if '`content`' is empty. To insert a preprocessing variable into the RSP output, use directive `<%@string name="⟨name⟩"%>`. There exist analogue directives for *numeric*, *integer* and *logical* variables.

```
<%@string name="page_format" content="article"%>
<%@string name="page_size" content="a4paper"%>

\documentclass[<%@string name="page_size"%>]{<%@string name="page_format"%>}
```

For convenience, there is a short format for setting a variable, which is `<%@string ⟨name⟩="⟨content⟩"%>`.

Remark: Just as for metadata, preprocessing variables transfer transparently between included child and parent documents when using `<%@include ⟨file⟩="⟨pathname|URL⟩"%>`.

2.5.4 Conditional inclusion/exclusion (`<%@if ...%>`)

Using so called RSP 'if-then-else' preprocessing directives, it is possible to include (exclude) parts of an RSP source document such that they appear (do not appear) in the output RSP product. The full syntax of an RSP 'if-then-else' variable is `<%@if test="⟨test⟩" name="⟨variable⟩" content="⟨value⟩" negate="⟨logical⟩"%>` ⟨include⟩ `<%@else%>` ⟨exclude⟩ `<%@endif%>`, where the `<%@else%>` statement is optional. The argument '`test`' specifies the type of test to conduct, '`name`' the variable name to be tested, and '`content`' the value to compare the variable to. The '`negate`' argument is optional and specifies whether the result of the test should be negated or not. The following tests are available (aliases in parentheses):

- `test="exists"`: check if a variable exists.
- `test="equal-to"` (`"=="`): check if a variable is equal to a specific value.
- `test="not-equal-to"` (`"!="`): check if a variable is not equal to a specific value.
- `test="less-than-or-equal-to"` (`"<="`): check if a variable is less than or equal to a specific value.
- `test="less-than"` (`"<"`): check if a variable is less than a specific value.
- `test="greater-than-or-equal-to"` (`">="`): check if a variable is greater than or equal to a specific value.
- `test="greater-than"` (`">"`): check if a variable is greater than a specific value.

For example,

```
<%@string name="version" content="devel"%>
<%@if test="exists" name="version"%>
 <%@if test="equal-to" name="version" content="devel"%>
This document presents methods that are under development.
 <%@else%>
This document presents methods that are well tested and stable.
 <%@endif%>
<%@else%>
Preprocessing variable 'version' was not set.
<%@endif%>
```

produces

```
This document presents methods that are under development.
```

If one removes the first line, then the output is

```
Preprocessing variable 'version' was not set.
```

For convenience, it is also possible to use a short format for testing a variable, which is `<%@if test="equal-to" ⟨name⟩="⟨value⟩"%>`.

2.5.4.1 If-then-else aliases (<%@ifeq ...%>)

For some of the common tests, there are also so called *alias directives*. They are:

- <%@ifeq ...%> is an alias for <%@if test="equal-to" ...%>.
- <%@ifneq ...%> is an alias for <%@if test="not-equal-to" ...%>.

Remark: Note that by combining the above aliases with the short format, one can write <%@ifeq A="42"%> instead of <%@if test="equal-to" name="A" content="42"%>.

2.6 Advanced

2.6.1 Compiling a standalone RSP document from a preprocessed but not evaluated RSP document template

However convenient it is to use a modularized RSP document that reuses many different RSP subdocuments, it may be tedious to share such as document with others. In order for someone else to compile the same RSP document, it is necessary to make sure all files being used are also shared with the receiver together with instructions where to put them. Some of these files may only be included under certain conditions, making the just add to the overall clutter at the receiver's end. In such cases, it may be useful to produce *a preprocessed but not yet evaluated RSP document* from the main RSP document. Such an RSP document will not contain any preprocessing directives and in that sense be self-contained and easier to share. It is also likely be easier for the receiver to follow. To compile such a stand-alone RSP document, do

```
> s <- rclean(file="random.txt.rsp")
```

This outputs a file preprocessed-random.txt.rsp in the current directory. Documents compiled this way will contain neither RSP comments nor RSP preprocessing directives. This may be useful if the source RSP subdocuments contain private comments and private parts that are conditional excluded during preprocessing.

3 Solutions specific to the R environment

3.1 Working with figures

Because the RSP language is context unaware, it does not have constructs for inserting content such as figures and tables. However, as we will show in this section, it is still very easy to create and include figures in the output document. We will first explain how to create image files and then show how they can be included in Markdown, HTML and LaTeX documents in a standard and unified way.

3.1.1 Creating image files

The toPDF() and toPNG() functions of the *R.devices* package are useful for creating image files to be included in for instance LaTeX and HTML. For example,

```
library("R.devices")

# Create objects with a bit thicker lines
devOptions("*", par=list(lwd=2))

toPNG("myFigure,yeah,cool", aspectRatio=0.6, {
```

```
  curve(dnorm, from=-5, to=+5)
})
```

creates a PNG image file named `myFigure,yeah,cool.png` whose height is 60% of its width and displays the Guassian density distribution. By default, the `toPDF()` and `toPNG()` functions write image files to the `figures/` directory. For more information, see `help("toPDF", package="R.devices")` and the vignettes of the *R.devices* package.

3.1.2 Including image files in Markdown

To dynamically create and include an image in a Markdown document, use the following RSP-embedded Markdown format:

```
![The Gaussian density function.](<%=toPNG("MyFigure,yeah,cool", aspectRatio=0.6, {
  curve(dnorm, from=-5, to=+5)
})%>)
```

which becomes Markdown syntax

```
![The Gaussian density function.](figures/MyFigure,yeah,cool.png)
```

When this Markdown document is post-processed into a HTML document (internally done using the *markdown* package), any image files will automatically be embedded as data URI strings in the final HTML document. Because such images are literally embedded inside the file, the HTML document becomes self-contained in the same was a a PDF document is self-contained.

3.1.3 Including image files in HTML

If you prefer to work with HTML directly, you can work with RSP-embedded HTML templates. Image files can be included in HTML documents either by linking to it via the images pathname or by embedding the content of the image files as a data URI. In order to use the first approach, do

```
<img src="<%=toPNG("MyFigure,yeah,cool", aspectRatio=0.6, {
  curve(dnorm, from=-5, to=+5)
})%>">
```

which results in

```
<img src="figures/MyFigure,yeah,cool.png">
```

To instead embedded the image as an Base64 encoded data URI, which has the advantage of making HTML document self-contained, set the following option

```
<% devOptions("png", field="dataURI") %>
```

which will cause the above 'MyFigure,yeah,cool.png' image file to be encoded and embedded as

```
<img src=" ... <truncated> ... K5CYII=">
```

To include, say, an SVG image file, just replace `toPNG()` with `toSVG()`. For other image formats, see the *R.devices* package.

3.1.4 Including image files in LaTeX

For LaTeX documents, one can embed figures using the following tidy RSP and LaTeX markup[12]

```
\includegraphics{<%=toPDF("MyFigure,yeah,cool", aspectRatio=0.6, {
  curve(dnorm, from=-5, to=+5)
})%>}
```

Since `toPDF()` returns the (relative) image pathname by default, the above will become

```
\includegraphics{figures/MyFigure,yeah,cool.pdf}
```

after the RSP document has been processed and the image file has been created. When this LaTeX document is compiled the result is as in in Figure 1.

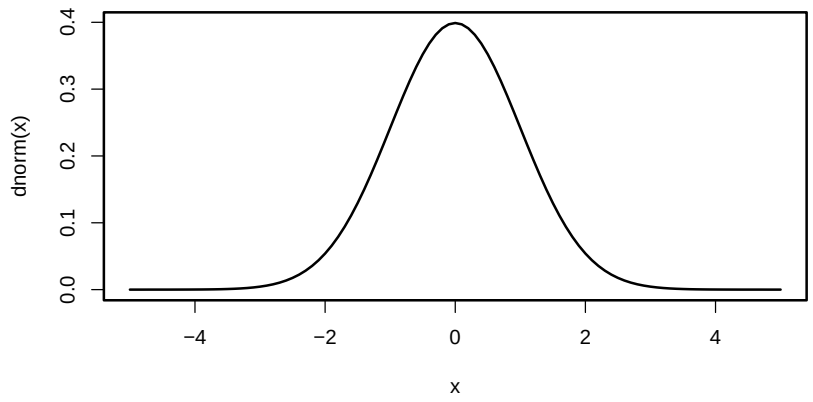

Figure 1: This figure was generated and inserted into the LaTeX document using RSP.

However, when inserting a figure in LaTeX it is recommended (not required) to do so without specifying neither the path nor the filename extension of the image files. Instead, one or more directories are specified at the beginning of the document telling LaTeX where to find images files, e.g.

```
\usepackage{graphicx}
\graphicspath{{figures/}{figures/external/}}
```

This tells LaTeX to search for image files in directory `figures/` and then in directory `figures/external/`. Next, we can include images as

```
\includegraphics{MyFigure,yeah,cool}
```

By leaving out the filename extension, LaTeX will automatically search[13] for image files of proper formats (typically PDF, EPS or PNG) depending on compiler settings, e.g. `*.png` and `*.pdf`. With this approach

[12] For readability, we have dropped the usual LaTeX commands for setting up the `figure` environment, centering and rescaling the image and creating the figure caption.

[13] In order for `\includegraphics{}` to locate the proper image file when leaving out the filename extension, the image file must not have other periods in the filename other than the one used for the extension.

it is easy to switch to use other image formats without having to edit the LaTeX document. On the other hand, this is already automatically take care of when using RSP. Regardless, in order to achieve this in RSP, all we need to do is to let *R.devices* use the so called *fullname* of the image file, that is the filename without extension, instead of the default (relative) *pathname*. This is achieved by setting

```
<% devOtions("*", field="fullname") %>
```

As a final remark, note that by replacing `toPDF()` with `toPNG()`, a PNG image file is inserted instead, which can dramatically reduce the size of the final PDF document, especially when scatter plots with a large number of data points are generated.

3.2 Including R code

3.2.1 Evaluating and capturing expression and output (<%= withCapture (⟨code chunk⟩)%>)

The `withCapture()` function of the *R.utils* package allows us to evaluate, embed and echo the text output of an R code chunk. Note that it is only standard output that is echoed, i.e. plots are *not* automatically embedded as figures. To include figures, see Section 3.1. Continuing, any formatting of the capture has to be done in the format of the output document. For instance, the following RSP-embedded LaTeX text

```
\begin{verbatim}
<%=withCapture({
for (kk in 1:3) {
  cat("Iteration #", kk, "\n", sep="")
}

type <- "horse";  # Comments are dropped
type
})%>
\end{verbatim}
```

produces

```
> for (kk in 1:3) {
+     cat("Iteration #", kk, "\n", sep = "")
+ }
Iteration #1
Iteration #2
Iteration #3
> type <- "horse"
> type
[1] "horse"
```

By adding arguments `code=FALSE` and/or `output=FALSE` to `withCapture()` one can control whether the deparsed R source code and/or the output of each subexpression is captured or not.

Remark: The *R.utils* package has to be attached for `withCapture()` to be available, e.g. via `<% library("R.utils") %>`.

Remark: The code is parsed and formatted by the R parser, meaning that indentation, empty lines, spacing and so on are *not* preserved when echoing this way. This is also why comments, semicolons and other R code constructs are dropped from the displayed code.

3.2.2 Future directions for including code

The `withCapture()` construct is rather rudimentary and has limitations on how the source code and echoed outputs are formatted. A much more powerful solution is then to utilize CRAN packages such as *formatR*, *evaluate* and *highlight*. The goal is to incorporate neat support for these in a future version of the *R.rsp* package.

4 History of RSP

The RSP markup language was first developed in May 2002. It was first inspired by the JavaServer Pages (JSP) markup languare available for the Java programming language which is used to create dynamic web content via for instance the Apache Tomcat web server. A related solution is the Active Server Pages (ASP) markup language for the VBScript language.

Initially the RSP engine was implemented as part of the *R.io* package (deprecated) but was moved to the *R.rsp* package in July 2005 and released on CRAN in July 2006. In early 2013, the RSP engine of *R.rsp* underwent a major redesign (compared to *R.rsp* v0.8.2 and before). This somewhat also affected the RSP language, but great efforts have been taken to keep it backward compatible.

The RSP language and the *R.rsp* engine has been in particularly heavy use by thousands of users of the Aroma Project (`http://aroma-project.org/`) as part of the generation of interactive Javascript and HTML reports. Several R packages are using RSP for their HTML and PDF vignettes, which has become straightforward since R v3.0.0.

In June 2007, the *brew* package (on CRAN) was introduced independently [private communication] of the *R.rsp* package. Although a different engine is used, the brew syntax is similar to the one in the RSP markup language.

Appendix

Session information

- R version 4.2.1 (2022-06-23), `x86_64-pc-linux-gnu`
- Locale: `LC_CTYPE=C.UTF-8`, `LC_NUMERIC=C`, `LC_TIME=C.UTF-8`, `LC_COLLATE=C.UTF-8`, `LC_MONETARY=C.UTF-8`, `LC_MESSAGES=C.UTF-8`, `LC_PAPER=C.UTF-8`, `LC_NAME=C`, `LC_ADDRESS=C`, `LC_TELEPHONE=C`, `LC_MEASUREMENT=C.UTF-8`, `LC_IDENTIFICATION=C`
- Running under: `Debian GNU/Linux bookworm/sid`
- Matrix products: default
- BLAS: `/usr/lib/x86_64-linux-gnu/blas/libblas.so.3.10.1`
- LAPACK: `/usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.1`
- Base packages: base, datasets, graphics, grDevices, methods, stats, utils
- Other packages: R.devices 2.17.1, R.methodsS3 1.8.2, R.oo 1.25.0, R.rsp 0.45.0
- Loaded via a namespace (and not attached): base64enc 0.1-3, compiler 4.2.1, digest 0.6.29, R.cache 0.15.0, R.utils 2.11.0, tools 4.2.1

This report was automatically generated using `rfile()` of the R.rsp package.